

HDOC DAY - 26

1. Algorithm

1. Start the program.
2. Display a menu containing the three right-aligned pattern options and an Exit option.
3. Read the user's choice.
4. Use a switch statement to perform the selected operation.
5. For **Right-aligned increasing triangle**:
 - a. Read the number of rows n.
 - b. For each row, print the required spaces followed by increasing numbers of *.
6. For **Right-aligned increasing number triangle**:
 - a. Read n.
 - b. For each row, print spaces followed by natural numbers from 1 to the row number.
7. For **Right-aligned decreasing triangle**:
 - a. Read n.
 - b. For each row, print the required spaces followed by decreasing numbers of *.
8. If the choice is 4, terminate the program.
9. For an invalid choice, display an error message.
10. Stop.

2. Test-Case Table

Test Case	Input	Expected Output	Actual Output	Result
TC1	Choice = 1, n = 4	Right-aligned increasing star triangle	* * * * * * * * * *	Pass

TC2	Choice = 2, n = 4	Right-aligned increasing number triangle	1 1 2 1 2 3 1 2 3 4	Pass
TC3	Choice = 3, n = 4	Right-aligned decreasing star triangle	* * * * * * * * * *	Pass
TC4	Choice = 1, n = 1	One right-aligned star	*	Pass
TC5	Choice = 2, n = 1	One right-aligned number	1	Pass
TC6	Choice = 3, n = 1	One star	*	Pass
TC7	Choice = 1, n = 0	Invalid number of rows	Invalid number of rows.	Pass
TC8	Choice = 5	Invalid choice message	Invalid choice.	Pass

The spaces in the expected output are used to create the right alignment.

3. C Program

```
#include <stdio.h>

/* Function for Right-aligned Increasing Triangle */
void rightIncreasingTriangle()
{
    int n, i, j;

    printf("Enter number of rows: ");
    scanf("%d", &n);

    if (n <= 0)
    {
        printf("Invalid number of rows.\n");
        return;
    }
}
```

```

for (i = 1; i <= n; i++)
{
    /* Print spaces */
    for (j = 1; j <= n - i; j++)
    {
        printf(" ");
    }

    /* Print stars */
    for (j = 1; j <= i; j++)
    {
        printf("* ");
    }

    printf("\n");
}
}

```

/* Function for Right-aligned Increasing Number Triangle */

```

void rightIncreasingNumberTriangle()
{
    int n, i, j;

    printf("Enter number of rows: ");
    scanf("%d", &n);

    if (n <= 0)
    {
        printf("Invalid number of rows.\n");
        return;
    }

    for (i = 1; i <= n; i++)
    {
        /* Print spaces */
        for (j = 1; j <= n - i; j++)
        {
            printf(" ");

```

```

    }

    /* Print numbers */
    for (j = 1; j <= i; j++)
    {
        printf("%d ", j);

    }

    printf("\n");
}
}

/* Function for Right-aligned Decreasing Triangle */
void rightDecreasingTriangle()
{
    int n, i, j;

    printf("Enter number of rows: ");
    scanf("%d", &n);

    if (n <= 0)
    {
        printf("Invalid number of rows.\n");
        return;
    }

    for (i = n; i >= 1; i--)
    {
        /* Print spaces */
        for (j = 1; j <= n - i; j++)
        {
            printf(" ");
        }

        /* Print stars */
        for (j = 1; j <= i; j++)
        {
            printf("* ");
        }
    }
}

```

```

    }

    printf("\n");
}
}

int main()
{
    int choice;

    do
    {
        printf("\n===== RIGHT-ALIGNED PATTERNS =====\n");
        printf("1. Right-aligned Increasing Triangle\n");
        printf("2. Right-aligned Increasing Number Triangle\n");
        printf("3. Right-aligned Decreasing Triangle\n");
        printf("4. Exit\n");

        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice)
        {
            case 1:
                rightIncreasingTriangle();
                break;

            case 2:
                rightIncreasingNumberTriangle();
                break;

            case 3:
                rightDecreasingTriangle();
                break;

            case 4:
                printf("Program terminated.\n");
                break;
        }
    } while (choice != 4);
}

```

```
        default:
            printf("Invalid choice.\n");
        }

    } while (choice != 4);

    return 0;
}
```

4. Sample Execution

```
===== RIGHT-ALIGNED PATTERNS =====
1. Right-aligned Increasing Triangle
2. Right-aligned Increasing Number Triangle
3. Right-aligned Decreasing Triangle
4. Exit
Enter your choice: 1
Enter number of rows: 4
      *
     * *
    * * *
   * * * *

===== RIGHT-ALIGNED PATTERNS =====
1. Right-aligned Increasing Triangle
2. Right-aligned Increasing Number Triangle
3. Right-aligned Decreasing Triangle
4. Exit
Enter your choice: 2
Enter number of rows: 4
      1
     1 2
    1 2 3
   1 2 3 4

===== RIGHT-ALIGNED PATTERNS =====
1. Right-aligned Increasing Triangle
2. Right-aligned Increasing Number Triangle
3. Right-aligned Decreasing Triangle
4. Exit
Enter your choice: 3
Enter number of rows: 4
   * * * *
  * * * *
 * * * *
* * * *

===== RIGHT-ALIGNED PATTERNS =====
1. Right-aligned Increasing Triangle
2. Right-aligned Increasing Number Triangle
3. Right-aligned Decreasing Triangle
4. Exit
```

5. Compile and Execute

Save the program as **testcase.c**.

Linux/macOS

```
gcc testcase.c -o testcase  
./testcase
```

Windows

```
gcc testcase.c -o testcase.exe  
testcase.exe
```

The program uses **nested loops, switch, separate parameterless void functions, and no arrays**, as required.